

```
from google.colab import files
uploaded = files.upload ()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Churn_Modelling.csv to Churn_Modelling (1).csv

```
import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("Churn_Modelling.csv")
```

```
df.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Ten
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   RowNumber             10000 non-null  int64
1   CustomerId            10000 non-null  int64
2   Surname               10000 non-null  object
3   CreditScore           10000 non-null  int64
4   Geography             10000 non-null  object
5   Gender               10000 non-null  object
6   Age                  10000 non-null  int64
7   Tenure               10000 non-null  int64
8   Balance              10000 non-null  float64
9   NumOfProducts        10000 non-null  int64
```

```
10 HasCrCard      10000 non-null  int64
11 IsActiveMember 10000 non-null  int64
12 EstimatedSalary 10000 non-null  float64
13 Exited          10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

```
df.describe ()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	I
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76488
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127648
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898

```
df.isnull().sum()
```

	0
RowNumber	0
CustomerId	0
Surname	0
CreditScore	0
Geography	0
Gender	0
Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCrCard	0
IsActiveMember	0
EstimatedSalary	0
Exited	0

dtype: int64

```
df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1, inplace=True)
```

```
df.columns = df.columns.str.lower()
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.drop_duplicates(inplace=True)
```

```
total_customers = len(df)
print(total_customers)
```

```
10000
```

```
churn_rate = (df['exited'].sum() / len(df)) * 100
```

```
print(round(churn_rate, 2))
```

20.37

```
avg_balance = df['balance'].mean()  
print(avg_balance)
```

76485.889288

```
avg_salary = df['estimatedsalary'].mean()  
print(avg_salary)
```

100090.239881

```
conn = sqlite3.connect ('bank_churn.db')
```

```
df.to_sql('customers', conn,  
if_exists = 'replace', index=False)
```

10000

```
query = """  
SELECT geography,  
COUNT(*) AS total_customers  
FROM customers  
GROUP BY geography  
"""  
pd.read_sql(query, conn)
```

	geography	total_customers
0	France	5014
1	Germany	2509
2	Spain	2477

```
query = """  
SELECT gender,  
AVG(exited) *100 AS churn_rate  
FROM customers  
GROUP BY gender  
"""  
pd.read_sql(query, conn)
```

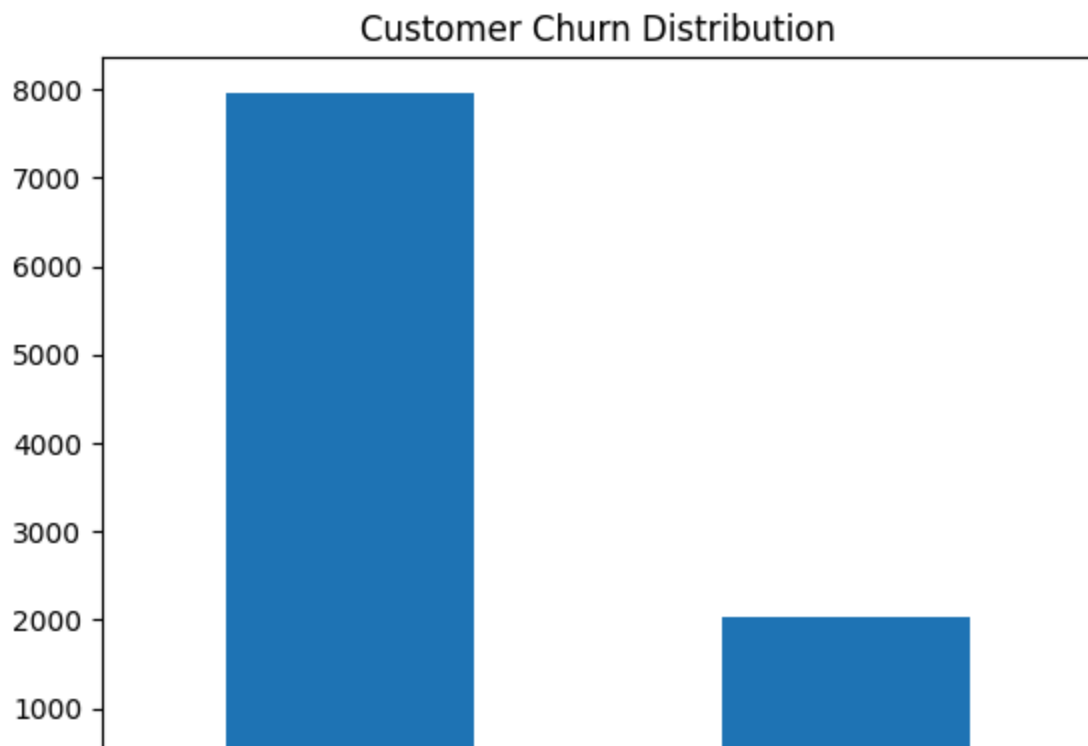
	gender	churn_rate
0	Female	25.071539
1	Male	16.455928

```
query = """
SELECT isactivemember,
COUNT(*) AS customers
FROM customers
GROUP BY isactivemember
"""
```

```
pd.read_sql(query, conn)
```

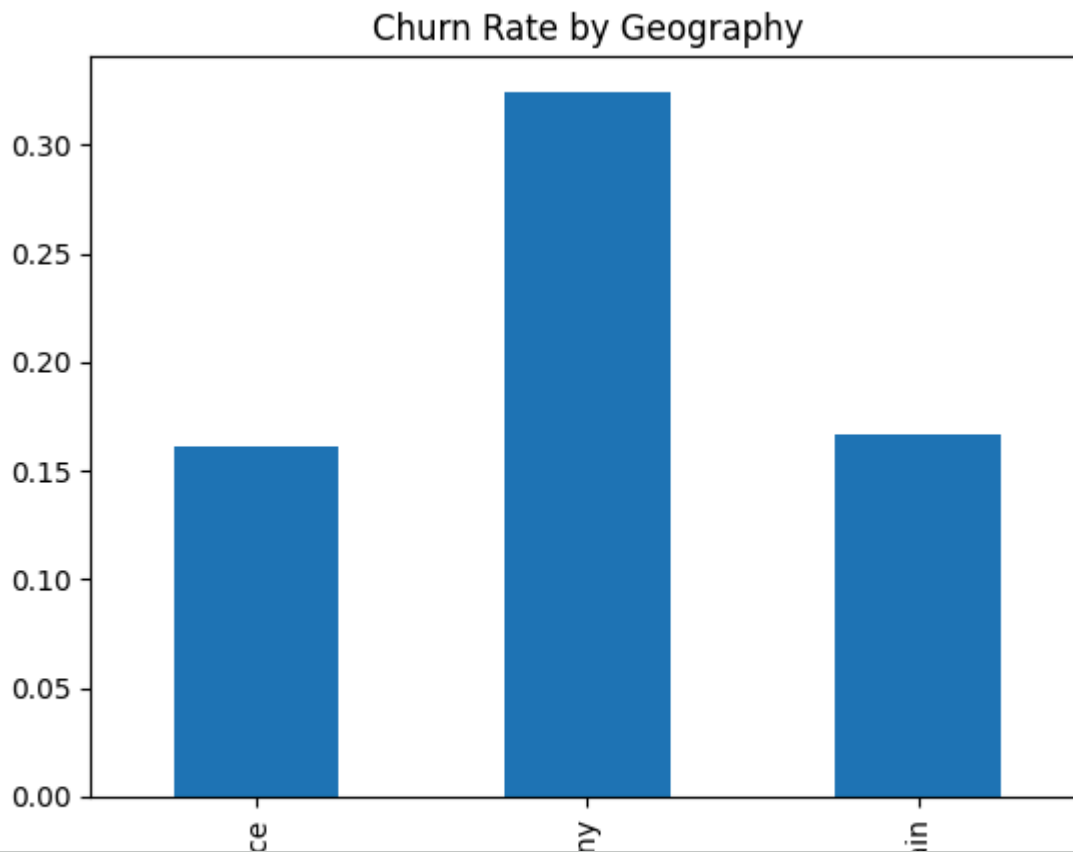
	isactivemember	customers
0	0	4849
1	1	5151

```
df['exited'].value_counts(). plot(kind = 'bar')
plt.title("Customer Churn Distribution")
plt.show()
```

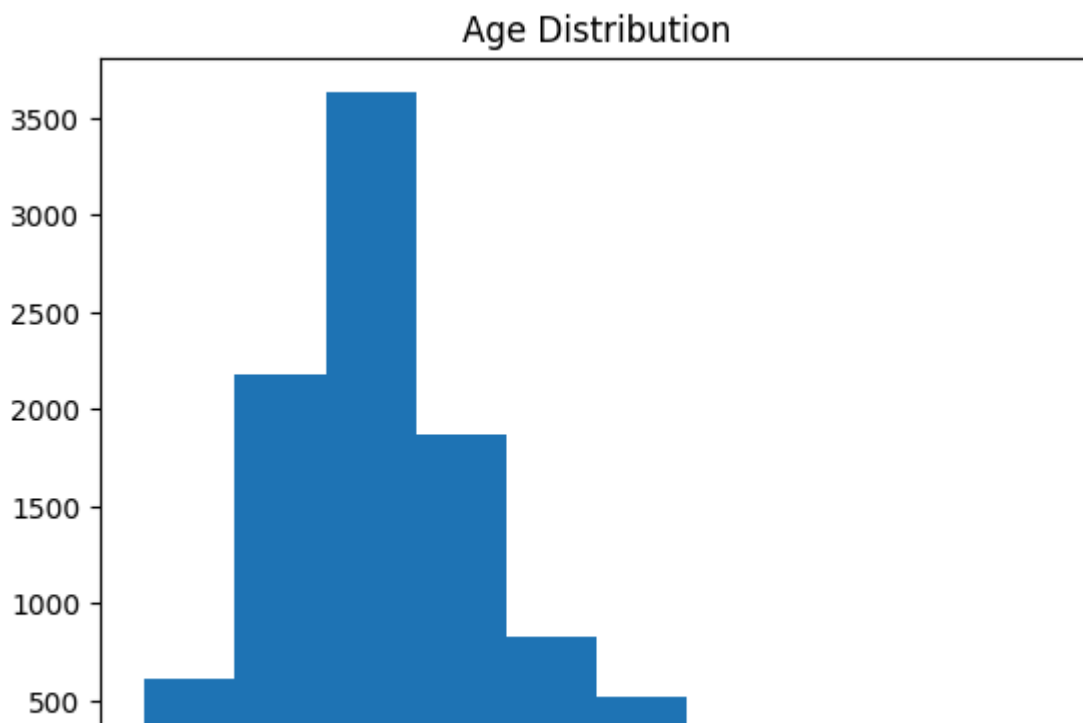


```
df.groupby('geography')['exited'].mean().plot(kind = 'bar')
plt. title("Churn Rate by Geography")
```

```
plt.show()
```



```
plt.hist(df['age'])  
plt.title("Age Distribution")  
plt. show()
```



```
df.to_csv("cleaned_churn_data.csv", index=False)
```

Tableau Visualizations:

1. Geography vs Churn
2. Gender vs Churn
3. Age Group vs Churn
4. Active vs Inactive Members
5. Credit Score Group vs Churn
6. Balance Category vs Churn

Business Insights

1. Germany shows the highest customer churn among all regions.
2. Senior customers are more likely to churn compared to younger customers.
3. Inactive members have significantly higher churn behavior.
4. Customers with low credit scores show greater churn risk.
5. Balance category analysis indicates customer retention differs by account value.
6. Female customers demonstrate slightly higher churn compared to male customers.

Recommendations:

- Improve retention campaigns.
- Focus on inactive users.
- Provide offers for high-risk customers.

Tools Used

- Python
- Pandas
- NumPy
- SQL
- Tableau
- Google Colab

Project Workflow

- 1. Data Collection
- 2. Data Cleaning